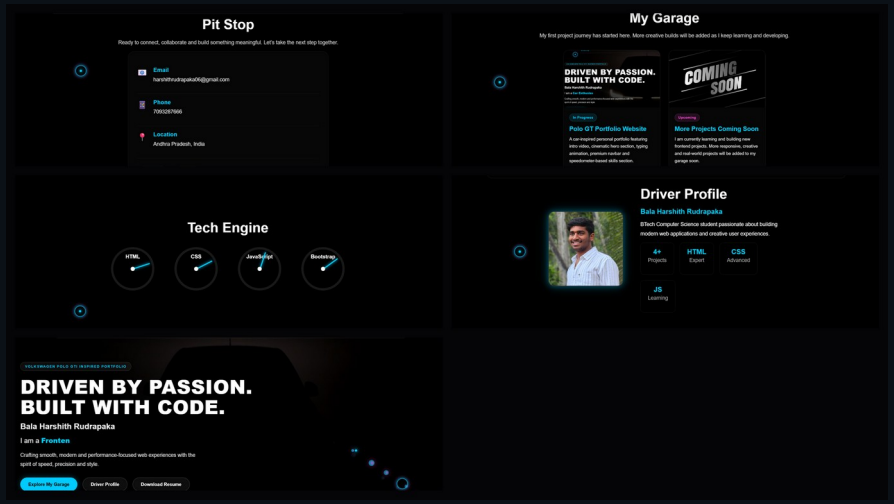
Slide 1 - Project Output Overview

This slide introduces the finished website and explains the overall idea before reading the code.

Code

```
Project Name: Polo GTI Garage Portfolio

Main idea:
- Car inspired personal portfolio
- Intro video at the start
- Premium dark navbar
- Hero section with typing text
- Driver profile section
- Speedometer skills section
- Garage projects section
- Pit Stop contact section
- Custom cursor and trail
- Reveal animations on scroll

Technologies Used:
HTML + CSS + JavaScript
```

Portfolio Screenshots



Explanation in Simple English

1. This project is a personal portfolio website made with a car theme.
2. The design idea is inspired by Volkswagen Polo GTI style and speed feeling.
3. The website opens with a video, so the user gets a strong first impression.
4. After the video ends, the main portfolio content becomes visible on the page.
5. The top part of the website has a glass style navigation bar.
6. The Home section has a background video and a bold heading.
7. The typing effect changes words automatically to show different roles.
8. The Driver Profile section introduces the student and skills in a simple way.
9. The Tech Engine section uses speedometer needles to show skill levels.
10. The Garage section shows current project work and future project plans.
11. The Pit Stop section is used as the contact area of the portfolio.
12. A custom cursor and light trail make the site feel more creative and modern.
13. Scroll based effects are used to reveal content smoothly when the user moves down.
14. This presentation explains how each part works with code and output together.
15. The goal is to make the whole project easy for a beginner to understand.

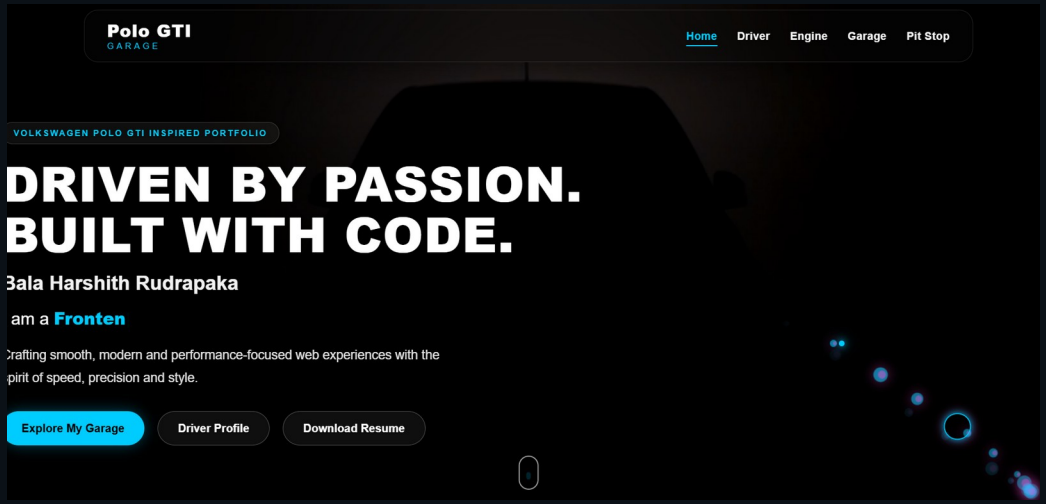
Slide 2 - HTML File Start and Main Layout

The HTML file is the skeleton of the website. It decides what appears on the page and in what order.

Code

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Harshith | Polo GT Garage</title>
  <link rel="stylesheet" href="aii3.css">
</head>
<body>
  <div id="intro-video">
    <video id="car-video" autoplay muted playsinline>
      <source src="../MY PORTIFOLIO/car animation.mp4" type="video/mp4">
    </video>
  </div>

  <div id="main-content">
    <!-- all portfolio sections are inside this -->
  </div>

  <script src="aii3.js"></script>
</body>
</html>
```

Related Output



Explanation in Simple English

1. The first line tells the browser that this is an HTML5 document.
2. The html tag wraps the whole web page from start to end.
3. The lang attribute says that the page language is English.
4. Inside the head tag, meta tags are used to support correct text and mobile display.
5. The title tag sets the text that appears on the browser tab.
6. The link tag connects the external CSS file to the HTML file.
7. That means all design and styling rules are loaded from aii3.css.
8. Inside the body tag, we first place the intro video container.
9. This intro video comes before the real content because it should show first.
10. The main-content div stores the full portfolio after the intro finishes.
11. This structure is very useful because JavaScript can hide and show sections easily.
12. The video tag uses autoplay, muted, and playsinline attributes for automatic play.
13. Muted is important because many browsers allow autoplay only when video is muted.
14. At the bottom, the script tag loads the JavaScript file after the HTML content.
15. This start layout is the foundation of the complete portfolio project.

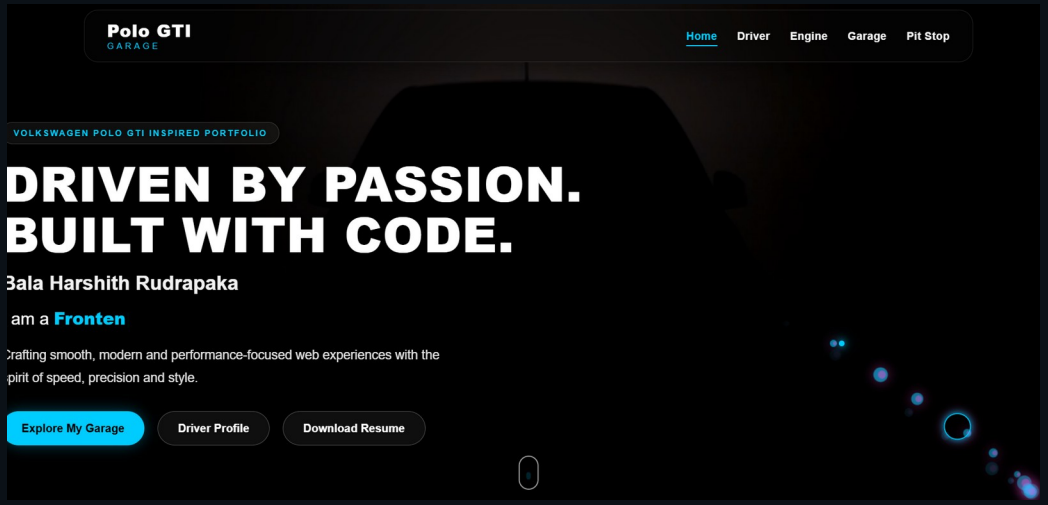
Slide 3 - HTML Navbar Structure

The navbar helps the user move to each section of the portfolio quickly.

Code

```
<nav class="navbar">
  <div class="nav-logo">
    <span class="logo-main">Polo GTI</span>
    <span class="logo-sub">Garage</span>
  </div>

  <ul class="nav-links">
    <li><a href="#Home">Home</a></li>
    <li><a href="#driver">Driver</a></li>
    <li><a href="#engine">Engine</a></li>
    <li><a href="#garage">Garage</a></li>
    <li><a href="#pitstop">Pit Stop</a></li>
  </ul>
</nav>
```

Related Output



Explanation in Simple English

1. The nav tag is used for navigation links at the top of the page.
2. The class navbar is used later in CSS to give the menu its premium style.
3. Inside the nav, one part is used for the logo text.
4. The text Polo GTI is shown as the main brand name of the portfolio.
5. The word Garage acts like a subtitle and supports the car theme.
6. The ul tag creates a list of navigation items.
7. Each li tag stores one link inside the navbar.
8. The href values start with a hash sign because they link to sections on the same page.
9. When the user clicks Home, the page scrolls to the section with id Home.
10. The same method is used for Driver, Engine, Garage, and Pit Stop sections.
11. This kind of one page navigation is common in portfolio websites.
12. It gives a smooth and modern feeling because the user stays on one page.
13. The navbar is also connected to JavaScript later for active link highlighting.
14. That means the current section link changes color while the user scrolls.
15. So this small HTML block plays a very important role in user experience.

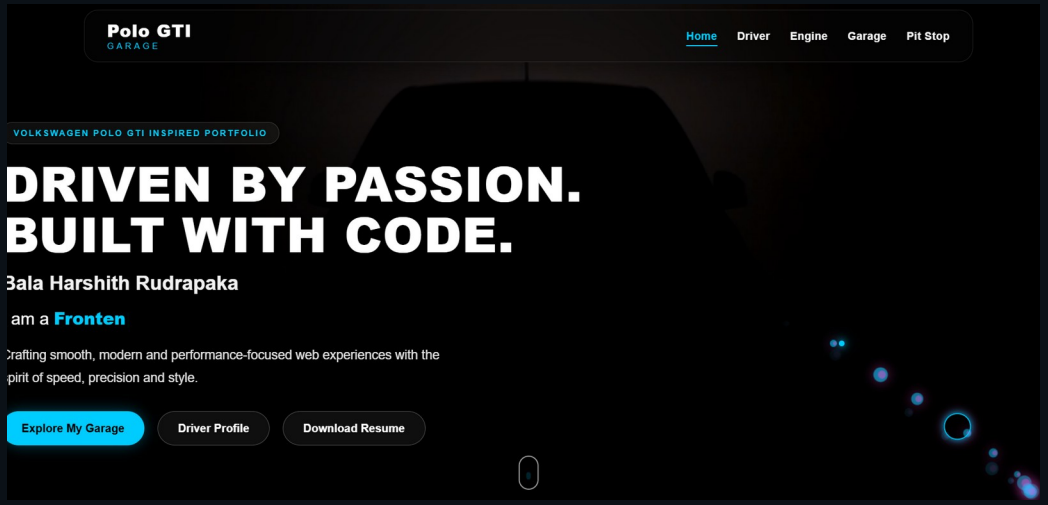
Slide 4 - HTML Home Section

The Home section is the hero section. It is the most important first content after the intro video.

Code

```
<section id="Home">
  <video autoplay muted loop class="Home-video">
    <source src="../MY PORTIFOLIO/gt-main.mp4" type="video/mp4">
  </video>

  <div class="Home-overlay"></div>

  <div class="content hero-content">
    <p class="mini-tag">VOLKSWAGEN POLO GTI INSPIRED PORTFOLIO</p>
    <h1 class="hero-title">Driven By Passion.<br>Built With Code.</h1>
    <h2 class="hero-name">Bala Harshith Rudrapaka</h2>
    <p class="typing-line">I am a <span id="typing-text"></span><span
class="cursor">|</span></p>
    <p class="hero-text">Crafting smooth, modern and performance-focused web
experiences...</p>
    <div class="hero-buttons">
      <a href="#garage" class="btn primary-btn">Explore My Garage</a>
      <a href="#driver" class="btn secondary-btn">Driver Profile</a>
      <a href="../MY PORTIFOLIO/RUDRAPAKA BALA HARSHITH RESUME(8).pdf" class="btn
secondary-btn" download>Download Resume</a>
    </div>
  </div>
```

Related Output



Explanation in Simple English

1. The section tag with id Home creates the main hero area of the website.
2. A background video is placed first so that it fills the full section behind the text.
3. The loop attribute means the video keeps playing again and again.
4. A dark overlay div is added to make the text easier to read on top of the video.
5. The content hero-content div holds the visible text and buttons.
6. The mini-tag line works like a small label to explain the portfolio theme.
7. The h1 tag is used for the main heading because it is the most important text here.
8. The h2 tag shows the student name clearly under the main heading.
9. The typing-line paragraph contains a span where JavaScript writes animated words.
10. The cursor span gives the small blinking line after the typing text.
11. The hero-text paragraph explains the purpose and style of the portfolio.
12. Three buttons are used so the user can explore work, read profile, or download resume.
13. The first button moves to the Garage section.
14. The second button moves to the Driver section and the third downloads the resume file.
15. This section mixes HTML structure with video, text, and actions in one strong opening screen.

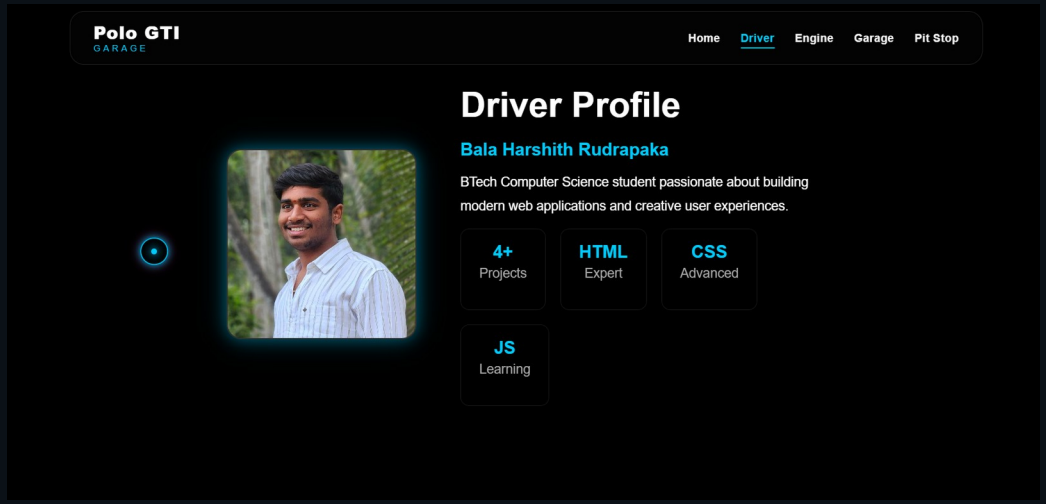
Slide 5 - HTML Driver Profile Section

This section introduces the student in a clean card style with photo and small stat blocks.

Code

```
<section id="driver">
  <div class="overlay"></div>
  <div class="driver-container">
    <div class="driver-image">
      <img src="../bg-img/bg2.jpeg">
    </div>

    <div class="driver-info">
      <h1>Driver Profile</h1>
      <h2>Bala Harshith Rudrapaka</h2>
      <p>BTech Computer Science student passionate about building modern web
applications...</p>

      <div class="driver-stats">
        <div class="stat"><h3>4+</h3><p>Projects</p></div>
        <div class="stat"><h3>HTML</h3><p>Expert</p></div>
        <div class="stat"><h3>CSS</h3><p>Advanced</p></div>
        <div class="stat"><h3>JS</h3><p>Learning</p></div>
      </div>
    </div>
  </div>
</section>
```

Related Output



Explanation in Simple English

1. The Driver Profile section acts like the About Me part of the portfolio.
2. An overlay is again used because the section background may have an image or dark theme.
3. The driver-container keeps the image and the text side by side.
4. The driver-image div holds the profile photo of the student.
5. The img tag loads the image from a local folder path.
6. The driver-info div stores all written information about the student.
7. A heading is added first to clearly name the section.
8. The second heading shows the full name in a highlighted style.
9. A short paragraph explains the student background and interest in development.
10. The driver-stats block is used to show quick information in small boxes.
11. One stat shows the number of projects built.
12. Other stat boxes show the student confidence level in HTML, CSS, and JavaScript.
13. This is a good idea because visitors can understand the profile quickly.
14. The structure is simple, but the layout becomes powerful after CSS styling is applied.
15. So this section mixes image, text, and mini cards in a very beginner friendly way.

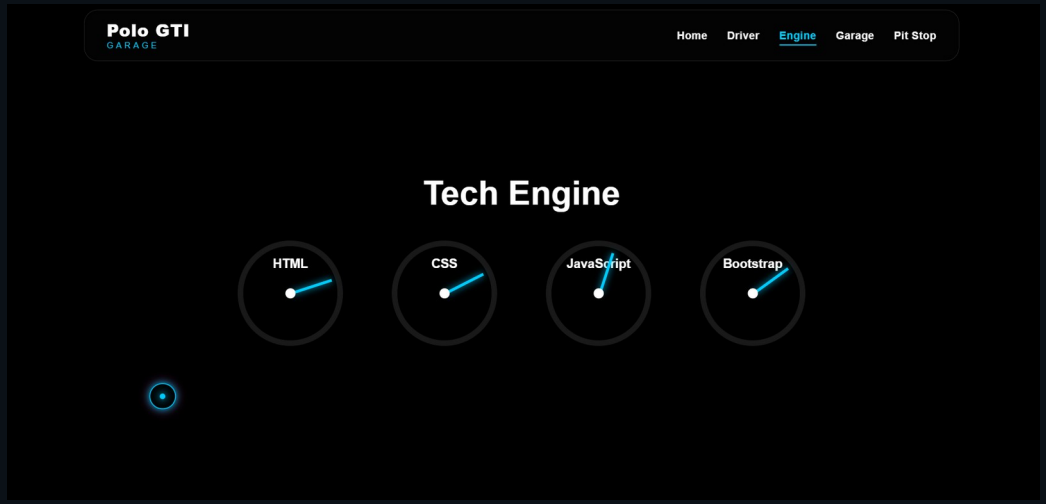
Slide 6 - HTML Tech Engine Section

This section uses speedometer style HTML so the skills match the car theme of the portfolio.

Code

```
<section id="engine">
  <div class="overlay"></div>
  <div class="content">
    <h1>Tech Engine</h1>

    <div class="speedometer-container">
      <div class="speed-card">
        <div class="speedometer" data-speed="90">
          <div class="needle"></div>
          <div class="center-circle"></div>
          <div class="speed-label">HTML</div>
        </div>
      </div>

      <div class="speed-card"> ... CSS 85 ... </div>
      <div class="speed-card"> ... JavaScript 60 ... </div>
      <div class="speed-card"> ... Bootstrap 80 ... </div>
    </div>
  </div>
</section>
```

Related Output



Explanation in Simple English

1. The engine section is designed differently from normal skill bars.
2. Instead of plain text percentages, the project uses circular speedometers.
3. This is a smart design decision because it matches the car theme of the portfolio.
4. The content starts with the heading Tech Engine.
5. The speedometer-container holds all skill meters in one row or wrapped layout.
6. Each speed-card is one separate box for one skill.
7. Inside each card, the speedometer div stores the needle and center circle.
8. The data-speed attribute stores the actual skill value like 90 or 85.
9. JavaScript later reads this value and rotates the needle.
10. The needle div is the moving line of the speedometer.
11. The center-circle div is the white round dot in the middle.
12. The speed-label tells the user what skill the speedometer belongs to.
13. By repeating the same HTML structure, the page can show many skills easily.
14. This is a good example of reusable HTML structure in project design.
15. The effect becomes complete only when CSS draws the circles and JS rotates the needle.

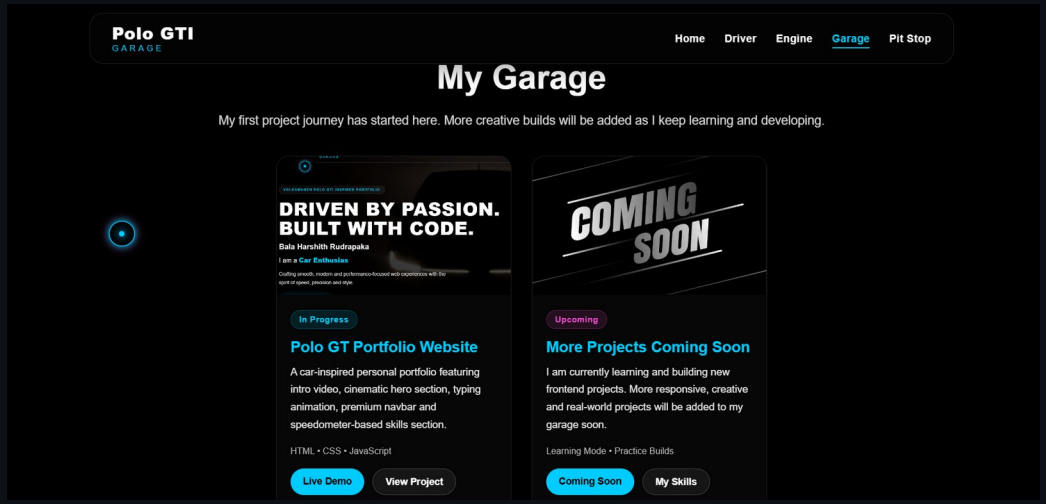
Slide 7 - HTML Garage Section

The Garage section is the project showcase area. It uses cards, images, badges, and buttons.

Code

```
<section id="garage">
  <div class="overlay"></div>
  <div class="content">
    <h1>My Garage</h1>
    <p class="garage-subtext">My first project journey has started here...</p>

    <div class="garage-cards">
      <div class="garage-card">
        <img src="../MY PORTIFOLIO/Screenshot 2026-03-15 163036.png" alt="Polo GT
Portfolio Project">
        <div class="garage-card-content">
          <span class="project-status in-progress">In Progress</span>
          <h3>Polo GT Portfolio Website</h3>
          <p>A car-inspired personal portfolio featuring intro video, cinematic hero
section...</p>
          <span class="tech-tag">HTML • CSS • JavaScript</span>
          <div class="garage-buttons">
            <a href="#" class="garage-btn live-btn">Live Demo</a>
            <a href="#" class="garage-btn code-btn">View Project</a>
          </div>
        </div>
      </div>
```

Related Output



Explanation in Simple English

1. The Garage section works like the Projects section of the portfolio.
2. The heading My Garage matches the vehicle theme and gives a unique feel.
3. The garage-subtext explains that this is the first project journey and more projects will come later.
4. The garage-cards div is used to hold multiple project cards together.
5. Each garage-card contains one project item.
6. The img tag is used to show the screenshot or preview image of that project.
7. The garage-card-content div stores text details under the image.
8. The project-status span acts like a small badge such as In Progress or Upcoming.
9. The h3 tag shows the project title clearly.
10. The paragraph below explains what the project does and what features it has.
11. The tech-tag line tells which technologies were used to build the project.
12. The garage-buttons div stores action buttons for demo or project viewing.
13. The second card is used as a placeholder for future work and learning progress.
14. This section is useful because it shows both current work and growth mindset.
15. Good project card HTML makes the portfolio feel more real and professional.

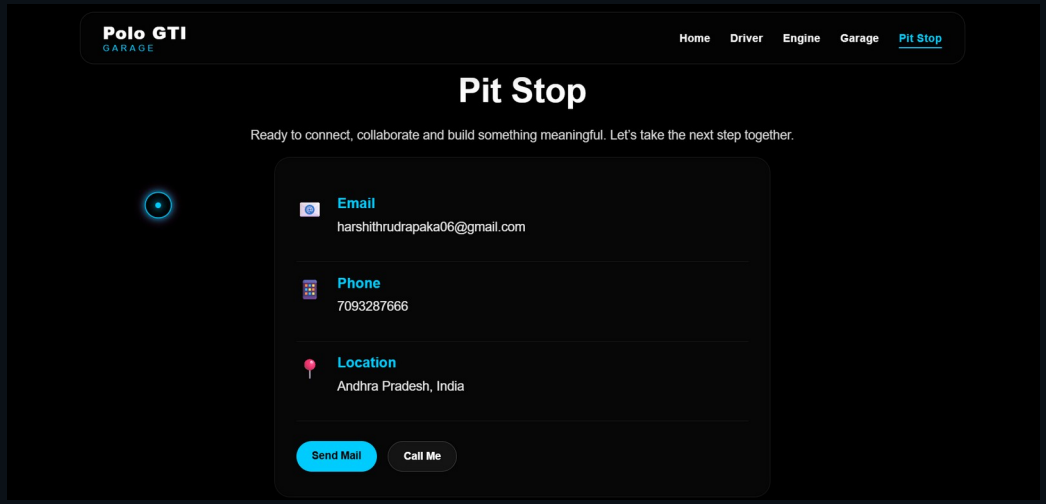
Slide 8 - HTML Pit Stop, Footer, Cursor and Script

This part finishes the HTML page with contact details, footer text, custom cursor elements and JavaScript link.

Code

```
<section id="pitstop">
  <div class="overlay"></div>
  <div class="content">
    <h1>Pit Stop</h1>
    <p class="pitstop-subtext">Ready to connect, collaborate and build something
meaningful...</p>
    <div class="pitstop-card">
      <div class="contact-item"><span
class="contact-icon">📧</span><div><h3>Email</h3><p>harshithrudrapaka06@gmail.com</p></
div></div>
      <div class="contact-item"><span
class="contact-icon">📱</span><div><h3>Phone</h3><p>7093287666</p></div></div>
      <div class="contact-item"><span
class="contact-icon">📍</span><div><h3>Location</h3><p>Andhra Pradesh,
India</p></div></div>
      <div class="pitstop-buttons">
        <a href="mailto:harshithrudrapaka06@gmail.com" class="pit-btn primary-pit-
btn">Send Mail</a>
        <a href="tel:7093287666" class="pit-btn secondary-pit-btn">Call Me</a>
      </div>
    </div>
  </div>
```

Related Output



Explanation in Simple English

1. The Pit Stop section is the contact section of the website.
2. The heading and subtitle invite the visitor to connect or collaborate.
3. The pitstop-card creates one main card that contains all contact details.
4. Each contact-item shows one type of information such as email, phone, or location.
5. The small emoji icons are used as simple contact symbols.
6. The email address is written inside a paragraph under its heading.
7. The phone number is shown in the next contact row.
8. The location row helps visitors understand where the student is based.
9. The first button uses mailto so it can open the email app directly.
10. The second button uses tel so it can call directly on supported devices.
11. The footer at the end gives closing text and copyright style information.
12. The cursor-dot and cursor-outline divs are extra HTML elements for the custom cursor effect.
13. The canvas element is added for the glowing trail effect behind the cursor.
14. Finally the script tag loads the JavaScript file that gives behavior to the whole project.
15. So this final HTML part connects contact, footer, animation, and interactivity together.

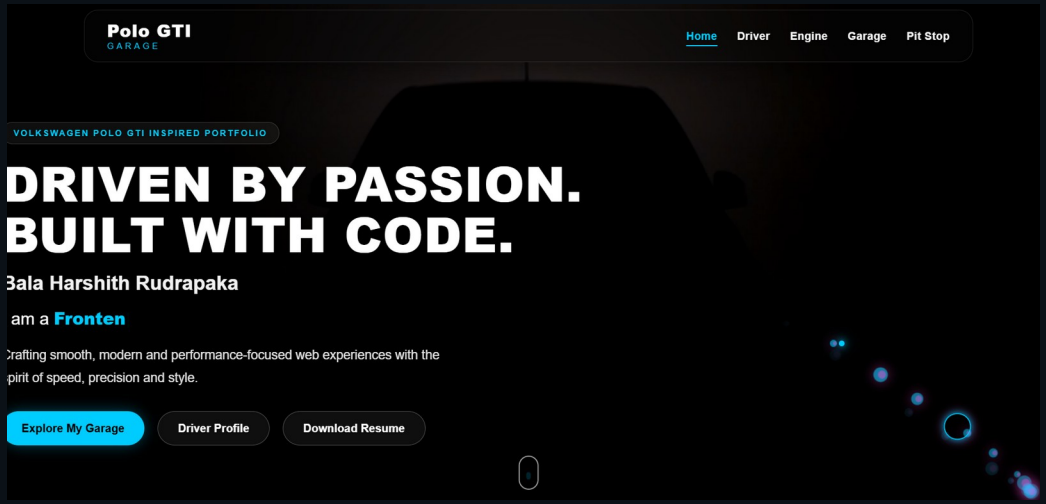
Slide 9 - CSS Global Reset, Body and Main Layout

CSS starts by cleaning default browser spacing and setting the dark theme of the whole website.

Code

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  font-family: Arial, sans-serif;
  background: #000;
  color: #fff;
  cursor: none;
  overflow-x: hidden;
}

a, button, .btn {
  transition: transform 0.35s ease, box-shadow 0.35s ease, color 0.35s ease, border-color
0.35s ease, background 0.35s ease;
}
```

Related Output



Explanation in Simple English

1. The star selector targets almost all elements on the page.
2. Margin and padding are set to zero to remove browser default spacing.
3. Box-sizing border-box makes width and height easier to manage in layouts.
4. The html selector uses smooth scroll so section links move nicely on click.
5. The body background is pure black to support the dark premium design.
6. The text color is set to white so the content is visible on the dark background.
7. The font-family sets the general text style of the whole page.
8. Cursor none hides the default mouse pointer because a custom cursor is used later.
9. Overflow-x hidden prevents unwanted horizontal scroll caused by animations or large elements.
10. The transition line on links and buttons creates smooth hover movement and color changes.
11. The intro-video container is fixed and covers the entire screen.
12. Its z-index is very high, so it stays above all other content at first.
13. The car-video style makes the intro video fill the screen without stretching badly.
14. Main-content is hidden at first because it should only appear after the intro ends.
15. These few CSS rules control the complete starting behavior and visual base of the project.

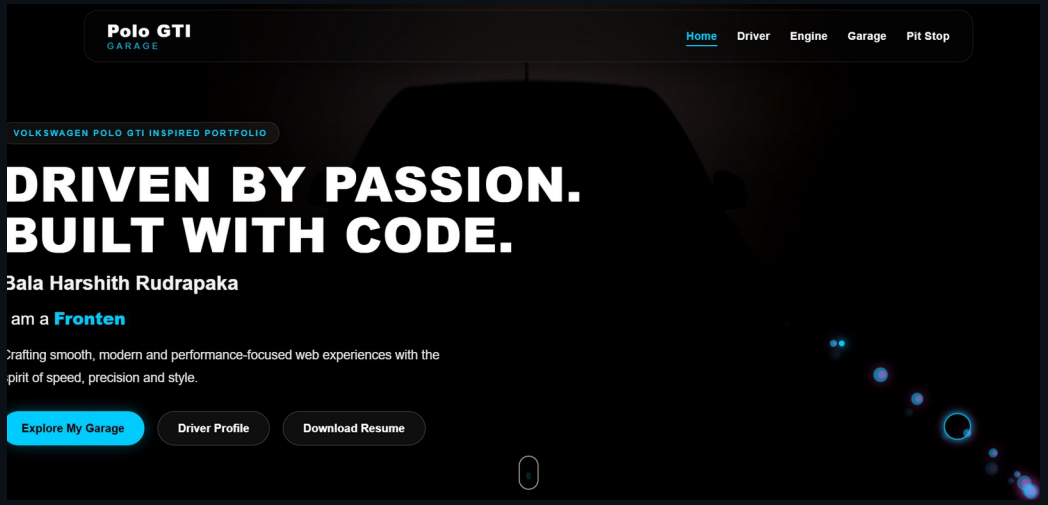
Slide 10 - CSS Premium Navbar Design

This CSS creates the glass look, spacing, and hover animation of the navigation bar.

Code

```
.navbar {
  position: fixed;
  top: 20px;
  left: 50%;
  transform: translateX(-50%);
  width: 92%;
  max-width: 1250px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 32px;
  background: rgba(10, 10, 10, 0.35);
  border: 1px solid rgba(255, 255, 255, 0.12);
  border-radius: 20px;
  backdrop-filter: blur(14px);
  box-shadow: 0 8px 30px rgba(0, 0, 0, 0.35);
  z-index: 1000;
}
.nav-links { display: flex; gap: 28px; list-style: none; }
.nav-links a { position: relative; text-decoration: none; color: #f5f5f5; font-size:
15px; font-weight: 600; }
.nav-links a::after { content: ""; position: absolute; left: 0; bottom: -6px; width: 0;
```

Related Output



Explanation in Simple English

1. The navbar is fixed so it stays visible while the user scrolls.
2. Top and left values place the navbar near the top center of the screen.
3. TranslateX is used to center the navbar correctly after using left 50 percent.
4. Width 92 percent gives side space on large screens and keeps the menu elegant.
5. Max-width stops the navbar from becoming too wide on very large displays.
6. Display flex places logo and links in one horizontal row.
7. Justify-content space-between pushes the logo left and the links right.
8. Padding adds breathing space inside the navbar box.
9. A semi transparent background gives the glass effect instead of a solid color.
10. Border and border-radius make the navbar look soft and premium.
11. Backdrop-filter blur is the key property that creates the modern glass feeling.
12. Box-shadow adds depth so the navbar stands out from the video background.
13. The nav-links list is displayed in a row using flex and gap spacing.
14. The ::after pseudo element creates the underline effect on hover.
15. So this CSS turns a simple HTML list into a polished premium navigation bar.

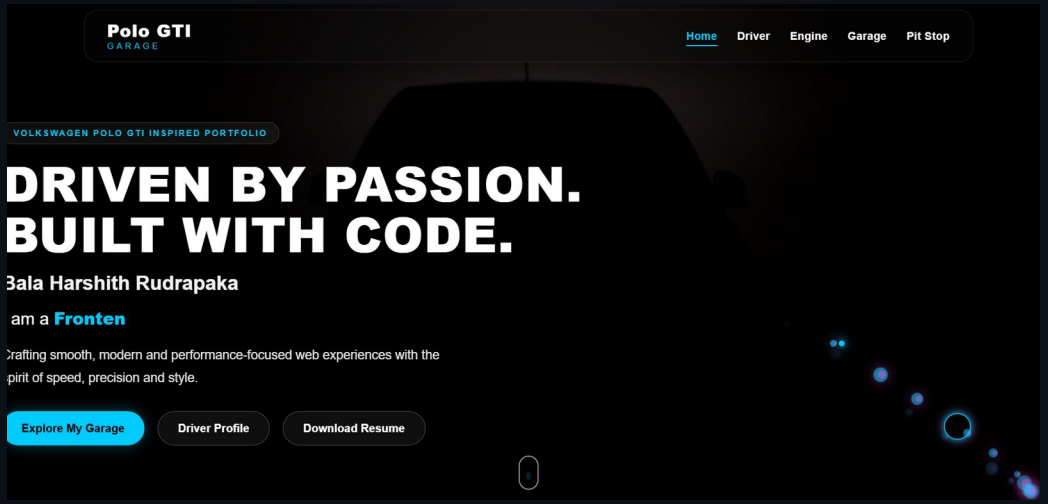
Slide 11 - CSS Home Hero Styling

This CSS controls the full screen hero area, overlay, text hierarchy, buttons, and scroll indicator.

Code

```
#Home {
  justify-content: flex-start;
  align-items: center;
  padding: 0 20px;
}
.Home-video {
  position: absolute; top: 0; left: 0;
  width: 100%; height: 100%;
  object-fit: cover; z-index: 0;
  filter: contrast(1.08) brightness(1.05);
}
.Home-overlay {
  position: absolute; inset: 0;
  background: linear-gradient(to right, rgba(0,0,0,0.68), rgba(0,0,0,0.24)),
              linear-gradient(to top, rgba(0,0,0,0.72), rgba(0,0,0,0.10));
  z-index: 1;
}
.hero-content { position: relative; z-index: 3; max-width: 1100px; text-align: left;
padding-top: 80px; }
.hero-title { font-size: 68px; line-height: 1.05; font-weight: 900; text-transform:
uppercase; }
.hero-name { font-size: 28px; font-weight: 700; color: #f3f3f3; }
```

Related Output



Explanation in Simple English

1. The Home section uses flex alignment so the hero content sits nicely on the screen.
2. Padding on the left and right keeps the text away from the screen edges.
3. The background video is absolutely positioned to cover the full hero section.
4. Object-fit cover makes the video fill the area without breaking its shape too much.
5. The filter property slightly improves video contrast and brightness.
6. The Home-overlay uses dark gradients to make text readable on the video.
7. Two gradients are combined to darken both the side and lower parts of the hero area.
8. Hero-content is placed above the video and overlay using a higher z-index.
9. A large max-width stops the text from becoming too stretched.
10. The hero-title uses very large font size and heavy weight to create strong impact.
11. The hero-name is smaller but still clear and easy to notice.
12. The typing line uses a medium size so it supports the heading without overpowering it.
13. The blue color on typing text matches the accent color of the theme.
14. Primary and secondary buttons are styled differently so users can understand main and secondary actions.
15. This CSS is what makes the Home section feel cinematic and premium.

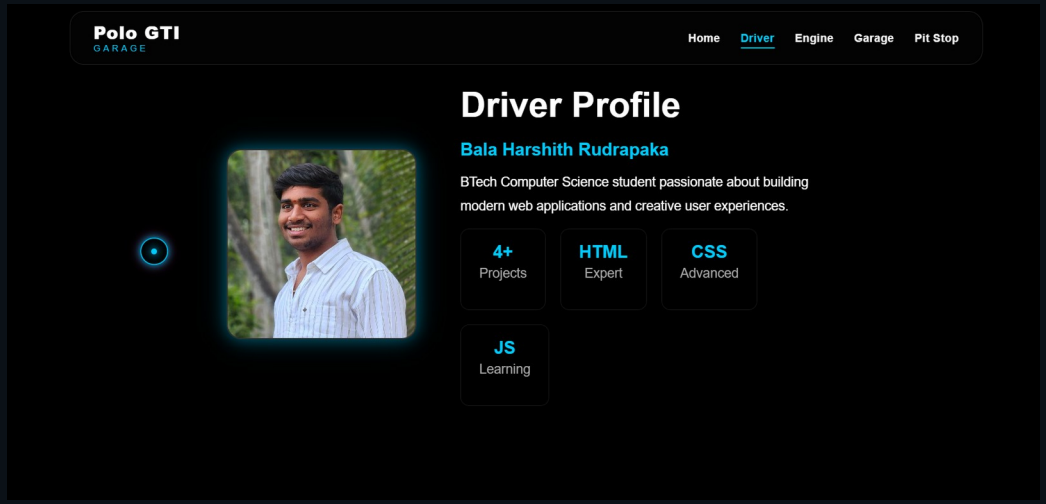
Slide 12 - CSS Driver Profile Styling

This CSS arranges the profile photo, text area, and stat cards into a clean responsive layout.

Code

```
.driver-container {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 60px;
  flex-wrap: wrap;
  position: relative;
  z-index: 3;
}
.driver-image img {
  width: 260px; height: 260px;
  object-fit: cover;
  border-radius: 20px;
  border: 2px solid rgba(255,255,255,0.2);
  box-shadow: 0 0 20px rgba(0,204,255,0.4), 0 0 40px rgba(0,204,255,0.2);
  transition: 0.4s;
}
.driver-image img:hover {
  transform: scale(1.05);
  box-shadow: 0 0 30px rgba(0,204,255,0.7), 0 0 60px rgba(255,79,216,0.4);
}
.driver-info { max-width: 500px; text-align: left; }
```

Related Output



Explanation in Simple English

1. The driver-container uses flex so the image and text can sit side by side.
2. Align-items center keeps both blocks aligned vertically in the middle.
3. Gap gives space between the photo and the information area.
4. Flex-wrap wrap helps the layout move nicely on smaller screens.
5. The image is given equal width and height so it becomes a neat square shape.
6. Object-fit cover ensures the image fills its box without distortion.
7. Border-radius makes the corners smooth and more modern.
8. A thin border and glow style match the blue accent color of the website.
9. Hover scale makes the profile photo feel interactive and alive.
10. The stronger hover shadow adds a premium neon look to the image.
11. Driver-info max-width stops the text area from becoming too wide to read.
12. Text-align left makes profile text easier to read than centered text.
13. Driver-stats uses flex and wrapping so small cards can align naturally.
14. Each stat card has padding, border, and dark background to look like dashboard blocks.
15. This CSS makes the profile section clear, attractive, and easy for users to scan.

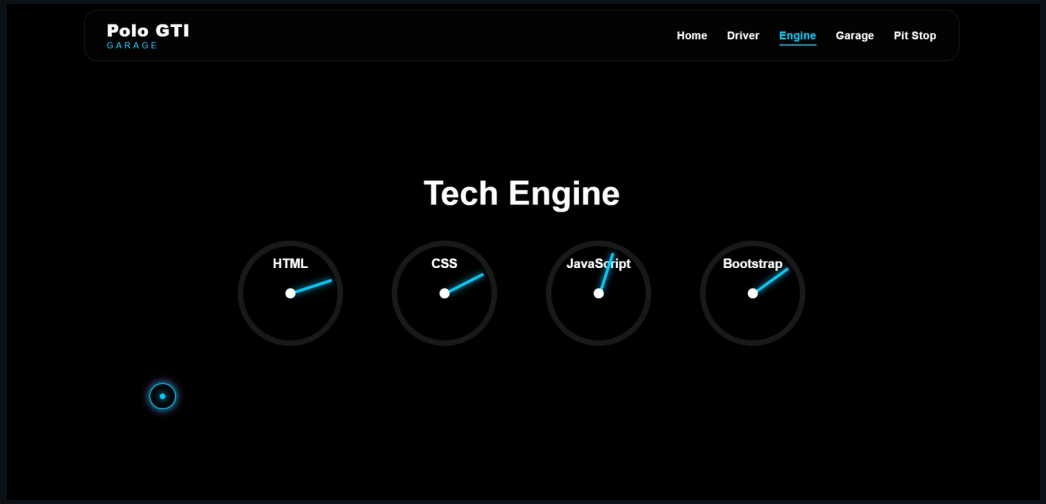
Slide 13 - CSS Engine Speedometer Styling

The speedometer design is made completely with CSS boxes, circles, and transforms.

Code

```
.speedometer-container {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 40px;
  margin-top: 40px;
}
.speed-card {
  width: 180px;
  text-align: center;
}
.speedometer {
  position: relative;
  width: 150px; height: 150px;
  border-radius: 50%;
  border: 8px solid rgba(255,255,255,0.1);
  margin: auto;
}
.needle {
  position: absolute;
  bottom: 50%;
  left: 50%;
```

Related Output



Explanation in Simple English

1. The speedometer-container uses flex so all meters can appear in one row.
2. Wrap is enabled so the meters can move to the next line on small screens.
3. Each speed-card keeps one meter centered in its own small area.
4. The speedometer box becomes a circle because border-radius is set to 50 percent.
5. A light border makes the outer ring of the speedometer visible.
6. The needle is absolutely positioned inside the circle.
7. Bottom 50 percent and left 50 percent start the needle from the center area.
8. Transform-origin bottom is very important because it lets the needle rotate from its base.
9. The default rotate -90deg points the needle to the starting side.
10. Transition 1.5 seconds makes the needle move smoothly when JavaScript changes it.
11. The center-circle is a white dot placed exactly in the middle of the speedometer.
12. Translate minus 50 percent is used to center the circle perfectly.
13. The speed-label is placed below the circular meter to show the skill name.
14. This CSS is simple but very smart because it creates a dashboard style without images.
15. The result is a skill section that matches the car theme in a very creative way.

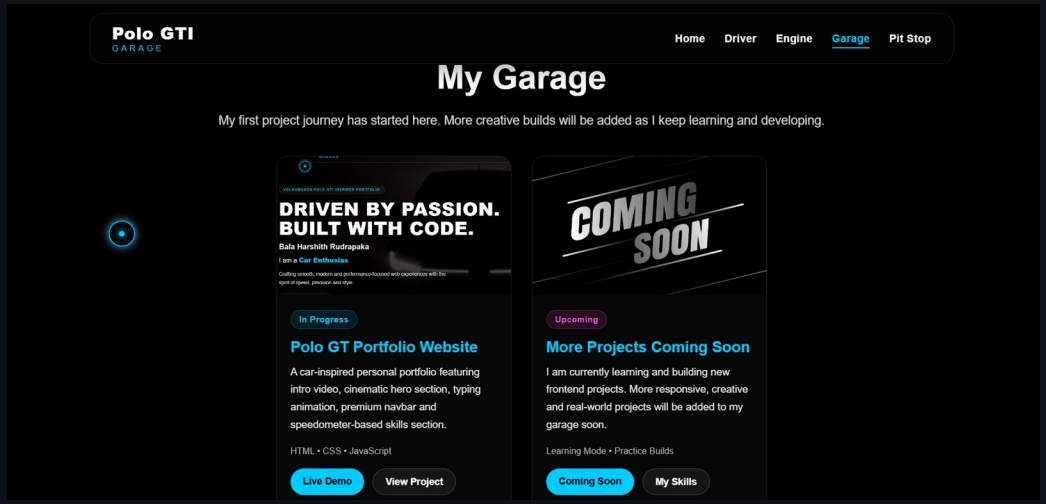
Slide 14 - CSS Garage Project Cards

This CSS gives the project cards their card layout, badges, image style, and button design.

Code

```
.garage-cards {
  display: flex;
  justify-content: center;
  gap: 30px;
  flex-wrap: wrap;
  margin-top: 20px;
}
.garage-card {
  width: 340px;
  background: rgba(10,10,10,0.72);
  border: 1px solid rgba(255,255,255,0.1);
  border-radius: 18px;
  overflow: hidden;
  box-shadow: 0 0 20px rgba(0,0,0,0.25);
  transition: 0.4s;
  backdrop-filter: blur(8px);
}
.garage-card:hover {
  transform: translateY(-10px) scale(1.02);
  box-shadow: 0 0 25px rgba(0,204,255,0.25);
}
.garage-card img { width: 100%; height: 200px; object-fit: cover; display: block; }
```

Related Output



Explanation in Simple English

1. The garage-cards wrapper uses flex to place project cards next to each other.
2. Gap gives equal space between the cards so the section feels neat and balanced.
3. The card width is fixed so the design stays consistent across projects.
4. The semi transparent dark background gives a modern glass card feeling.
5. A light border helps the card edges stay visible on the dark background.
6. Border-radius makes the cards look smooth and polished.
7. Overflow hidden is important so the image corners follow the card shape.
8. The box-shadow gives depth and separates the card from the page background.
9. Hover transform lifts the card slightly to make the design interactive.
10. The image is made to fully fill the card top using width 100 percent and object-fit cover.
11. Project-status badges are styled as small rounded pills for clear status indication.
12. In-progress and upcoming use different colors so the meaning is easy to notice.
13. Live and code buttons are styled differently to create action priority.
14. This kind of project card CSS is very common in real portfolio websites.
15. It makes the Garage section look complete, honest, and professional.

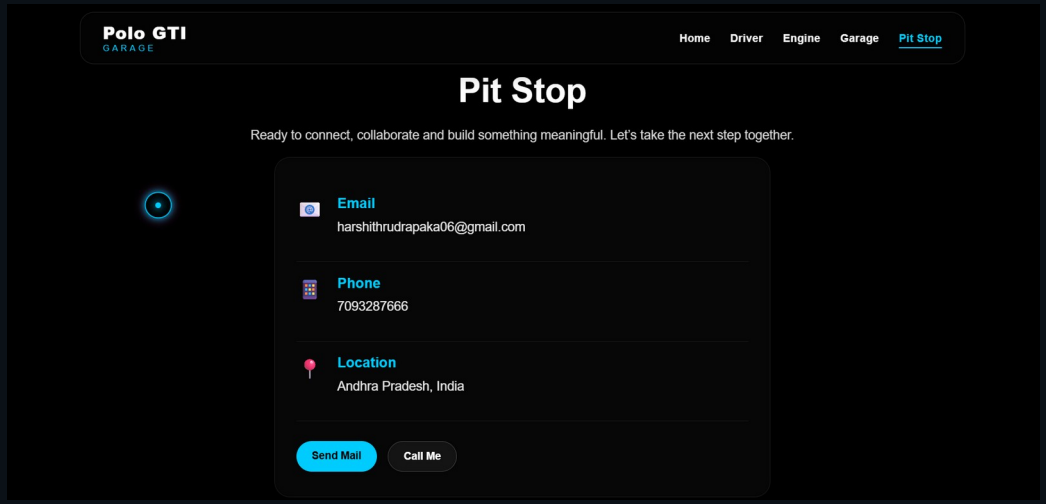
Slide 15 - CSS Pit Stop, Footer, Reveal and Responsive

This slide shows how the contact card, footer, reveal animation, and mobile adjustments are styled.

Code

```
.pitstop-card {
  max-width: 700px;
  margin: 0 auto;
  padding: 35px 30px;
  border-radius: 22px;
  background: rgba(10,10,10,0.65);
  border: 1px solid rgba(255,255,255,0.08);
  backdrop-filter: blur(10px);
  box-shadow: 0 0 25px rgba(0,0,0,0.28);
  text-align: left;
}
.contact-item { display: flex; align-items: flex-start; gap: 18px; padding: 18px 0;
border-bottom: 1px solid rgba(255,255,255,0.08); }
.pit-btn { padding: 12px 22px; border-radius: 30px; font-size: 15px; font-weight: 700; }
.footer { text-align: center; padding: 22px 15px; background: #050505; border-top: 1px
solid rgba(255,255,255,0.08); }
#driver .content, #engine .content, #garage .content, #pitstop .content {
  opacity: 0;
  transform: translateY(50px);
  transition: all 1s ease;
}
#driver .content.show-reveal, #engine .content.show-reveal, #garage .content.show-reveal,
```

Related Output



Explanation in Simple English

1. The pitstop-card has a fixed maximum width so the contact area does not become too wide.
2. Margin auto keeps the card centered in the section.
3. Padding creates enough breathing room inside the card.
4. Border radius and blur give the same premium glass style used in other sections.
5. Each contact item is displayed in a row using flex.
6. The gap helps the icon and text stay separated clearly.
7. A border line under each contact item creates visual grouping and order.
8. The pit buttons are styled as rounded action buttons similar to the hero buttons.
9. The footer gets a simple dark background and top border so it feels like a clear ending.
10. Reveal CSS starts section content with zero opacity and lower position.
11. When the class show-reveal is added, the content fades in and moves up smoothly.
12. This makes scrolling feel more dynamic and professional.
13. The media query is used to adjust layout for screens smaller than 768 pixels.
14. On mobile, the custom cursor and trail are hidden because they are not useful on touch devices.
15. Responsive CSS is important because a portfolio should work well on both laptop and mobile.

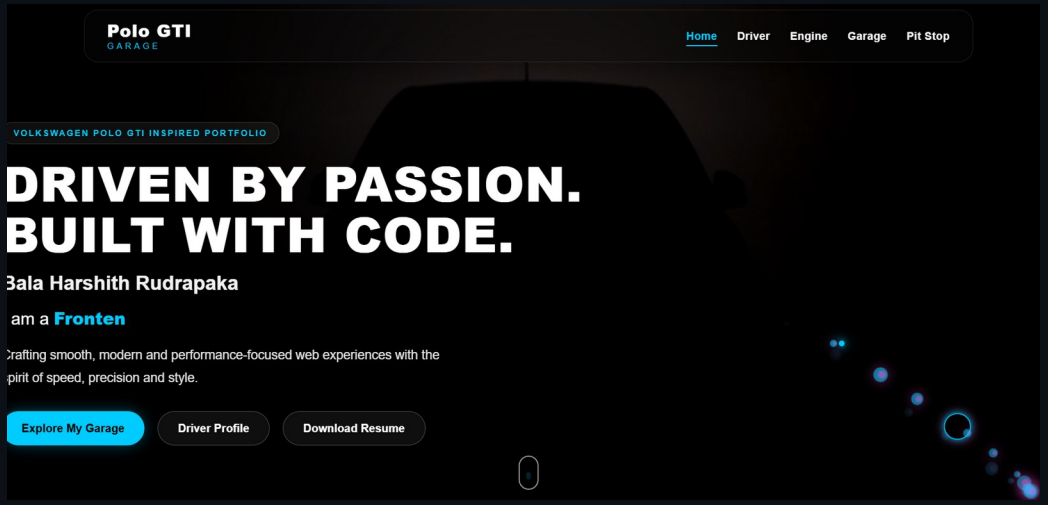
Slide 16 - JavaScript Intro Video and Typing Animation

The first JavaScript part controls the intro video and the changing text in the Home section.

Code

```
const introVideo = document.getElementById("car-video");

introVideo.onended = function () {
  document.getElementById("intro-video").style.display = "none";
  document.getElementById("main-content").style.display = "block";
};

const words = [
  "Full Stack Developer",
  "Frontend Developer",
  "Creative Web Designer",
  "Car Enthusiast"
];

let wordIndex = 0;
let charIndex = 0;
let isDeleting = false;

function typeEffect() {
  const typingText = document.getElementById("typing-text");
  if (!typingText) return;
  const currentWord = words[wordIndex];
```

Related Output



Explanation in Simple English

1. The first line selects the intro video element from the HTML page.
2. The onended event runs only when the video has fully finished playing.
3. Inside that event, the intro-video container is hidden with display none.
4. Then main-content is shown with display block so the portfolio appears.
5. This is how the page changes from intro screen to real content automatically.
6. The words array stores the list of roles that should appear in the typing effect.
7. WordIndex keeps track of which word is active right now.
8. CharIndex keeps track of how many letters should be shown at the moment.
9. The variable isDeleting helps the function know whether it should type or erase.
10. TypeEffect is a custom function that runs again and again using setTimeout.
11. Substring is used to show only part of the current word.
12. When the full word is typed, the code waits for a moment before deleting.
13. When all letters are deleted, the code moves to the next word in the array.
14. The modulo operator helps the words repeat in a loop from the beginning.
15. So this JavaScript creates both the intro reveal and the dynamic typing animation.

```
  setTimeout(typeEffect, isDeleting ? 50 : 100);
}
```

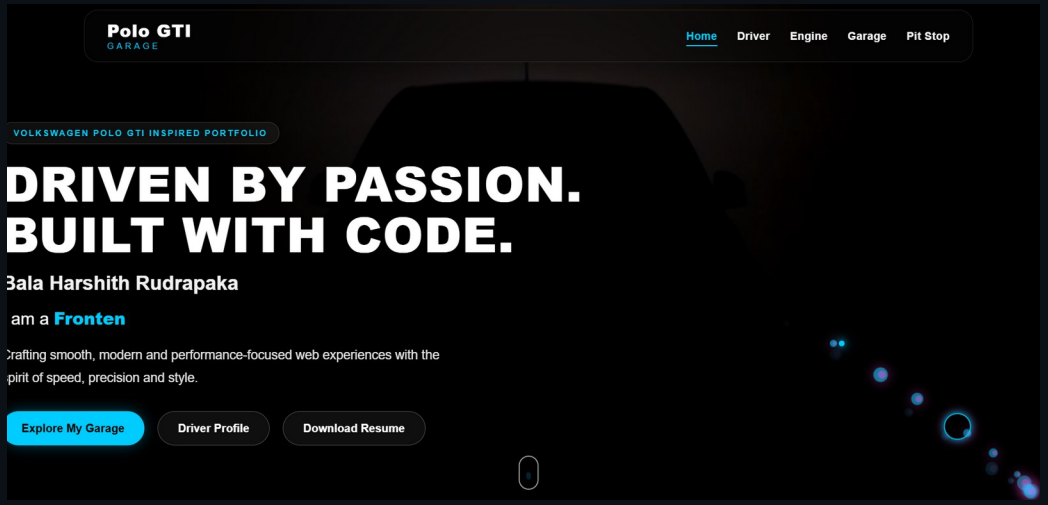
Slide 17 - JavaScript Custom Cursor and Canvas Trail

This code hides the normal cursor and replaces it with a glowing dot, outline, and moving light trail.

Code

```
const dot = document.querySelector(".cursor-dot");
const outline = document.querySelector(".cursor-outline");

let mouseX = 0;
let mouseY = 0;
let outlineX = 0;
let outlineY = 0;

document.addEventListener("mousemove", (e) => {
  mouseX = e.clientX;
  mouseY = e.clientY;
  dot.style.left = mouseX + "px";
  dot.style.top = mouseY + "px";
});

function animateOutline() {
  outlineX += (mouseX - outlineX) * 0.15;
  outlineY += (mouseY - outlineY) * 0.15;
  outline.style.left = outlineX + "px";
  outline.style.top = outlineY + "px";
  requestAnimationFrame(animateOutline);
}
```

Related Output



Explanation in Simple English

1. The dot and outline elements are selected from the HTML page first.
2. MouseX and mouseY store the current real position of the mouse.
3. OutlineX and outlineY store the delayed position for the smooth follow effect.
4. The mousemove event runs whenever the user moves the mouse.
5. Inside that event, the real mouse position is updated from the browser event.
6. The small inner dot is moved directly to the exact mouse position.
7. AnimateOutline is a function that moves the outer ring more slowly.
8. The formula with 0.15 makes the outline follow the mouse with easing.
9. RequestAnimationFrame keeps calling the function again before each screen repaint.
10. This creates a smooth animation instead of a jumpy movement.
11. Canvas is selected to draw the glowing trail behind the cursor.
12. The getContext method gives the 2D drawing area of the canvas.
13. Each mouse move adds a small object to the trails array.
14. Every object stores x position, y position, life value, and size value.
15. So this JavaScript creates the custom cursor system that gives the site a unique feel.

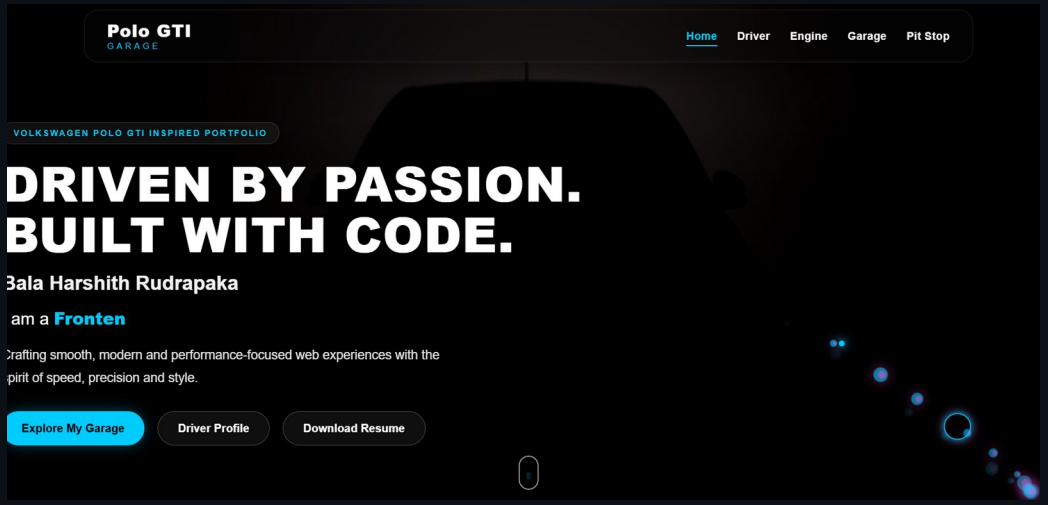
Slide 18 - JavaScript Canvas Drawing Logic

This part actually draws and removes the glowing trail circles on the canvas.

Code

```
function resizeCanvas() {
  canvas.width = window.innerWidth;
  canvas.height = window.innerHeight;
}
resizeCanvas();
window.addEventListener("resize", resizeCanvas);

function animateTrail() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  for (let i = trails.length - 1; i >= 0; i--) {
    const t = trails[i];

    ctx.beginPath();
    ctx.arc(t.x, t.y, t.size, 0, Math.PI * 2);
    ctx.fillStyle = `rgba(0, 204, 255, ${t.life / 40})`;
    ctx.shadowBlur = 18;
    ctx.shadowColor = "rgba(255, 79, 216, 0.7)";
    ctx.fill();

    ctx.beginPath();
    ctx.arc(t.x + 2, t.y, t.size * 0.55, 0, Math.PI * 2);
```

Related Output



Explanation in Simple English

1. ResizeCanvas makes the canvas size match the current browser window size.
2. This is important because a canvas does not automatically resize correctly by itself.
3. The function is called once at the start so the trail has correct width and height.
4. It is also attached to the resize event for when the browser window changes size.
5. AnimateTrail is the function that draws the glowing trail continuously.
6. ClearRect removes the previous frame so old drawings do not stay forever.
7. The loop moves backward through the trails array so items can be removed safely.
8. Each trail object is stored in the variable t for easy reading.
9. BeginPath starts a new shape before drawing the first glowing circle.
10. Arc draws the circle using the x, y, and size values stored in the trail object.
11. FillStyle uses a template string so the opacity changes with the remaining life value.
12. ShadowBlur and shadowColor create the glowing neon effect around the circle.
13. A second smaller circle is drawn with a pink tone to make the effect richer.
14. Life is reduced every frame and dead trail objects are removed from the array.
15. RequestAnimationFrame keeps this drawing process running smoothly on every frame.

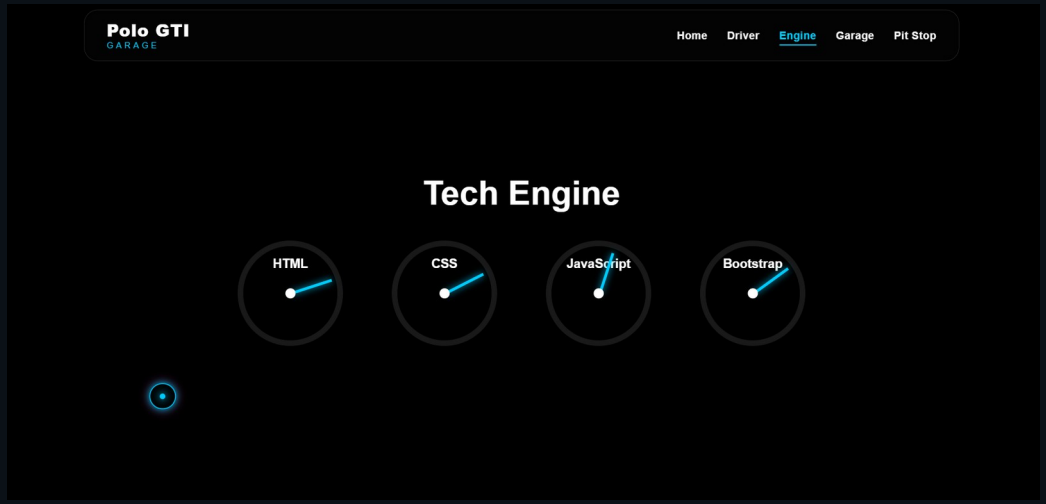
Slide 19 - JavaScript Speedometer, Reveal and Active Navbar

The last JavaScript block controls scroll based animations and active section behavior.

Code

```
const speedometers = document.querySelectorAll(".speedometer");
let engineAnimated = false;

function animateSpeedometers() {
  const engineSection = document.getElementById("engine");
  const sectionTop = engineSection.getBoundingClientRect().top;

  if (sectionTop < window.innerHeight && !engineAnimated) {
    speedometers.forEach((meter) => {
      const speed = meter.getAttribute("data-speed");
      const needle = meter.querySelector(".needle");
      const degree = (speed / 100) * 180 - 90;
      setTimeout(() => { needle.style.transform = "rotate(" + degree + "deg)"; }, 300);
    });
    engineAnimated = true;
  }
}
window.addEventListener("scroll", animateSpeedometers);

const revealElements = document.querySelectorAll("#driver .content, #engine .content,
#garage .content, #pitstop .content");
function revealOnScroll() {
```

Related Output



Explanation in Simple English

1. All speedometer elements are selected together using querySelectorAll.
2. The variable engineAnimated is used like a switch to stop repeated needle animation.
3. AnimateSpeedometers checks the position of the engine section using getBoundingClientRect.
4. If the engine section enters the visible screen area, the animation can begin.
5. For each speedometer, the code reads the data-speed value from the HTML.
6. Then it finds the needle inside that specific speedometer.
7. The degree formula converts the speed percentage into a rotation angle.
8. SetTimeout adds a small delay so the movement feels smoother to the user.
9. The scroll event is used because the animation should happen only when the user reaches the section.
10. RevealOnScroll selects the content blocks of multiple sections and watches their position.
11. When a block comes close to the visible area, the class show-reveal is added.
12. That class works together with CSS to fade in and move up the section content.
13. The final scroll event checks which section is currently active on the page.
14. Then the matching navbar link gets the active-link class and changes color.
15. This is a great example of JavaScript improving both motion and navigation feedback.

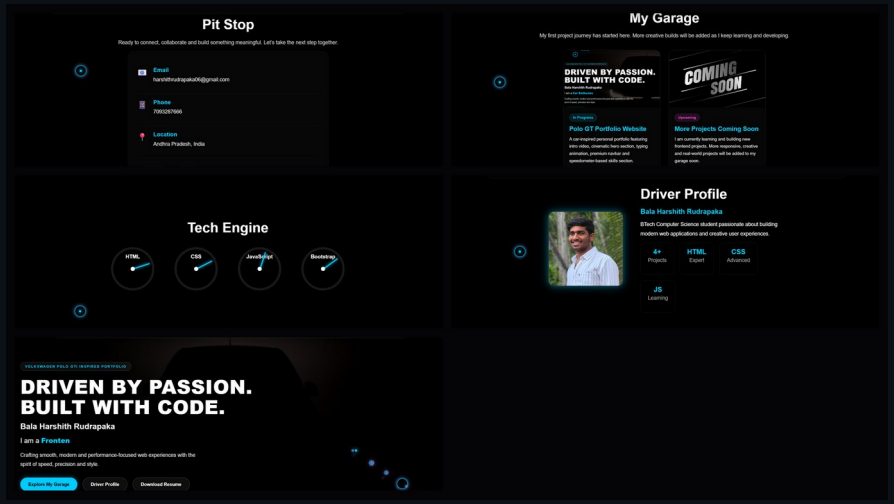
Slide 20 - Final Project Summary for Beginners

This closing slide explains what a beginner can learn by studying this portfolio project.

Code

```
What this portfolio teaches:

HTML:
- Page structure
- Sections, links, buttons, video, cards

CSS:
- Layout with flexbox
- Dark theme design
- Hover effects
- Responsive design
- Animation with transitions

JavaScript:
- DOM selection
- Events
- Typing effect
- Video end logic
- Mouse animation
- Canvas drawing
- Scroll animation
```

Final Output Summary



Explanation in Simple English

1. This project is a very good practice example for a beginner web developer.
2. The HTML part teaches how to divide one page into meaningful sections.
3. It also teaches how to use video, images, headings, buttons, and contact details.
4. The CSS part teaches how to create layout, spacing, colors, shadows, and responsive design.
5. It also shows how to build a premium dark theme with one consistent color style.
6. The JavaScript part teaches event handling and DOM manipulation in a real project.
7. The intro video logic shows how one screen can change into another screen automatically.
8. The typing effect teaches how to update text step by step using JavaScript.
9. The custom cursor teaches how to react to mouse movement in a creative way.
10. The canvas trail teaches how to draw graphics on the page frame by frame.
11. The speedometer section teaches how HTML data and JavaScript can work together.
12. The reveal effect teaches how scrolling can be used to trigger animation.
13. The active navbar teaches how to connect section positions with user interface feedback.
14. A beginner who understands this project will improve in both frontend logic and design thinking.
15. So this portfolio is not just a project output, it is also a full learning exercise.